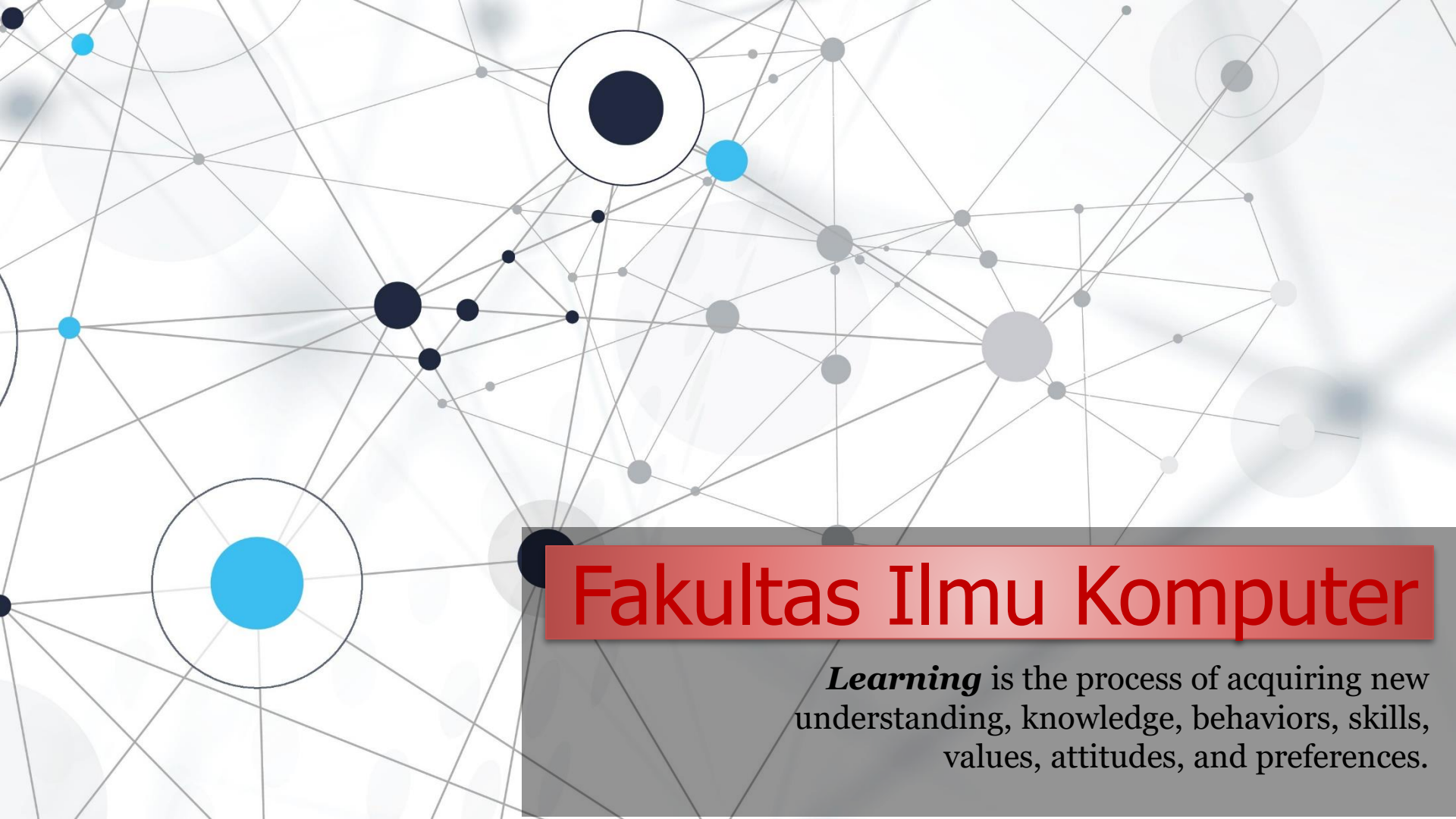


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

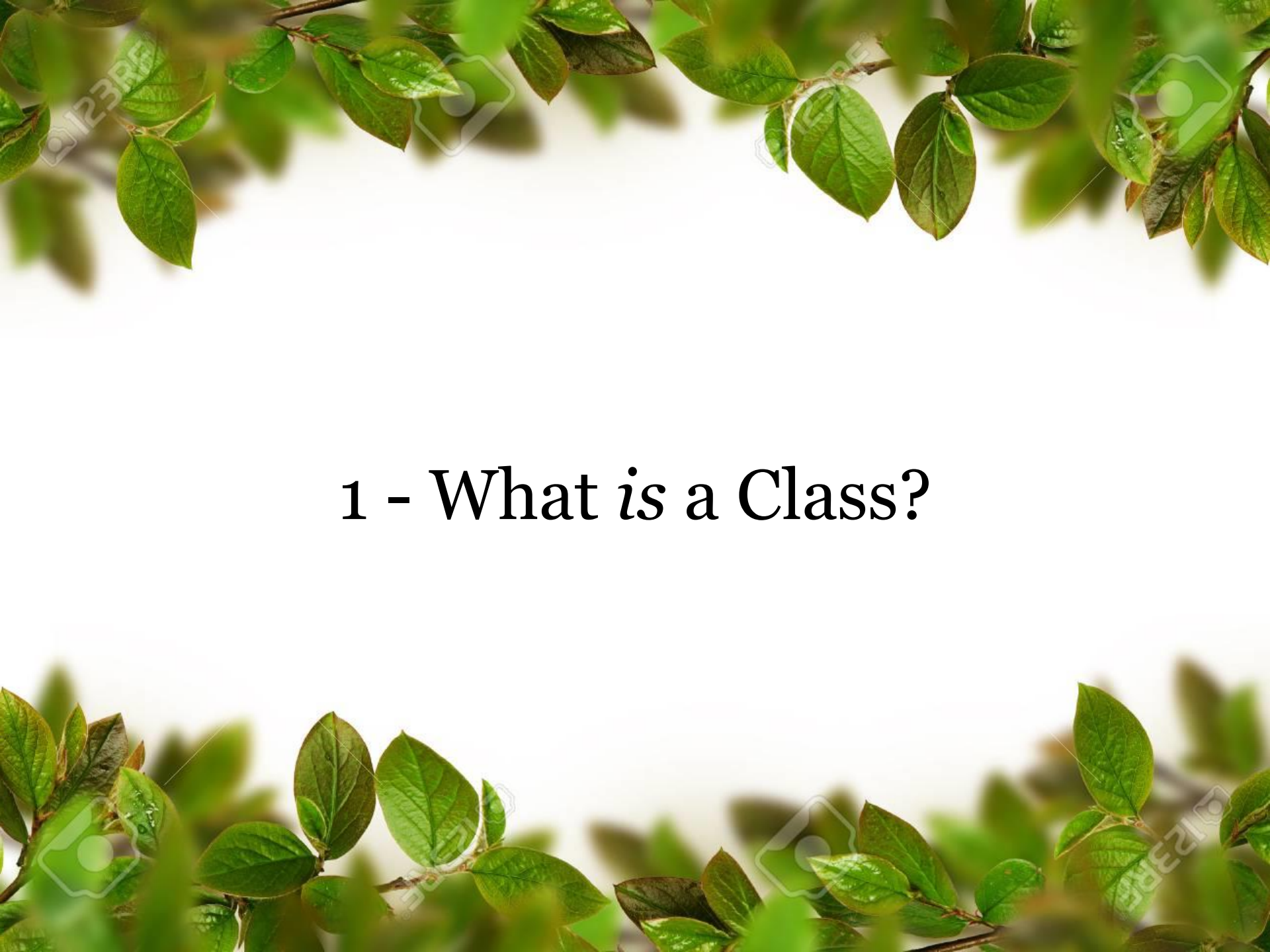
Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

OOP in Python – Classes/Objects ***(Parent Classes, Child Classes, Instance Attributes)***



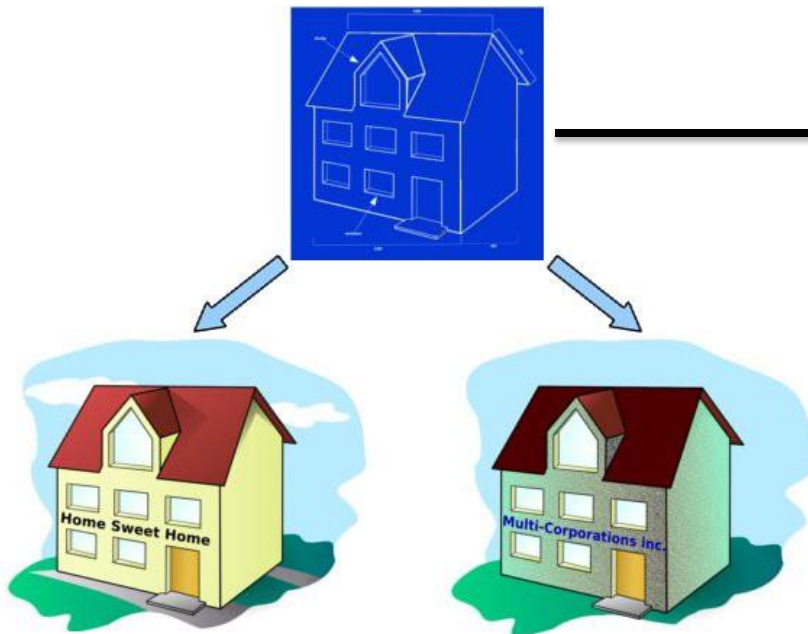
**Welcome to *the* Object Oriented
Programming *course***

The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green with some brownish edges, suggesting they might be from a specific type of tree or shrub. The text is centered in the middle of the slide.

1 - What is a Class?

What is a Class?

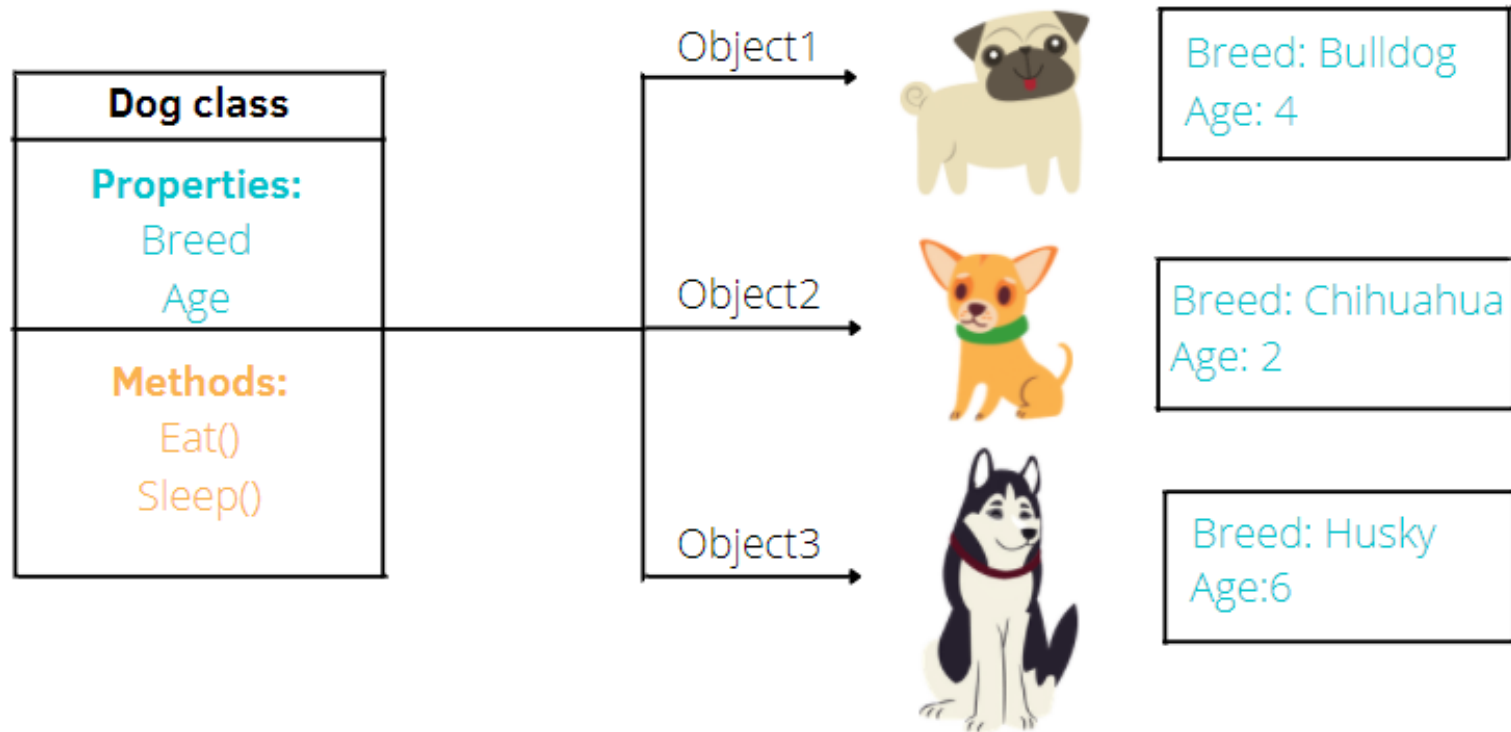
- 1) A **user-defined prototype** (**blueprint**, **design**) for an object that defines a set of attributes that characterize any object of the class. The **attributes** are **data members (class variables and instance variables)** and methods, accessed via dot notation.
- 2) A **Class in Python** is a **logical grouping of data and functions**. It gives the freedom to create **data structures** that contains arbitrary (*yg berubah-ubah, dynamic*) content and hence easily accessible.



Class, **Prototype**,
blueprint, **design**

Class atau kelas-kelas adalah *prototipe* untuk wadah menghimpun data dan kebergunaan yang kemudian menghasilkan objek. Setiap class baru akan menghasilkan objek baru yang kemudian bisa dibuat instance dengan memiliki atribut yang ada.

What is a Class?

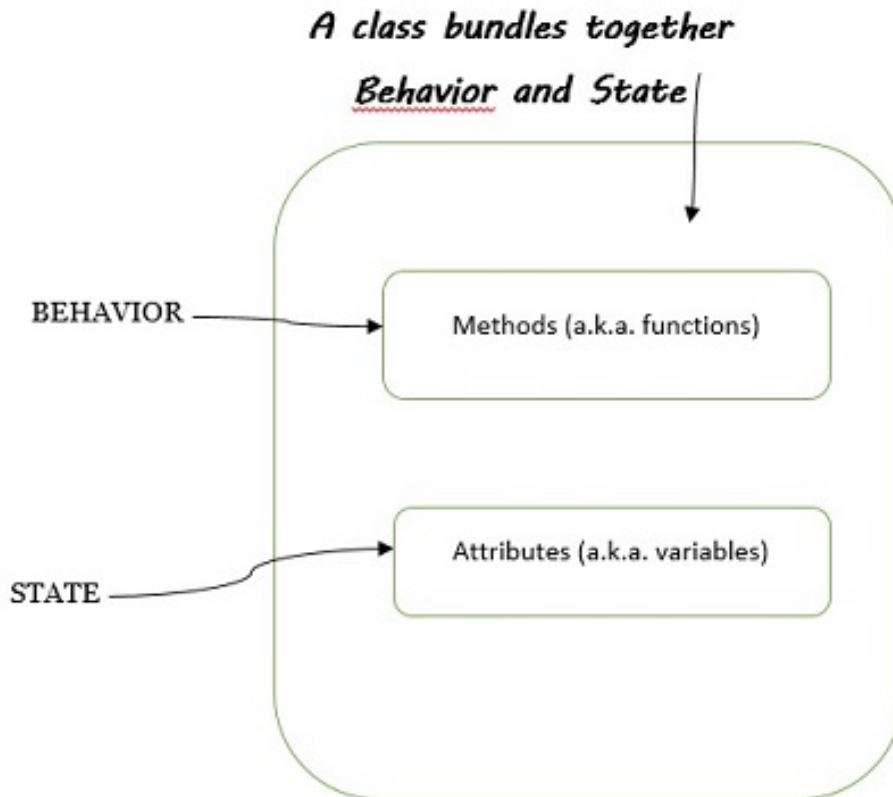


What is instance in Python?

Dalam pemrograman berorientasi objek (OOP) di Python, istilah "instance" merujuk kepada **OBJECT** yang dibuat dari suatu **CLASS**.

Class Bundles : Behavior and State

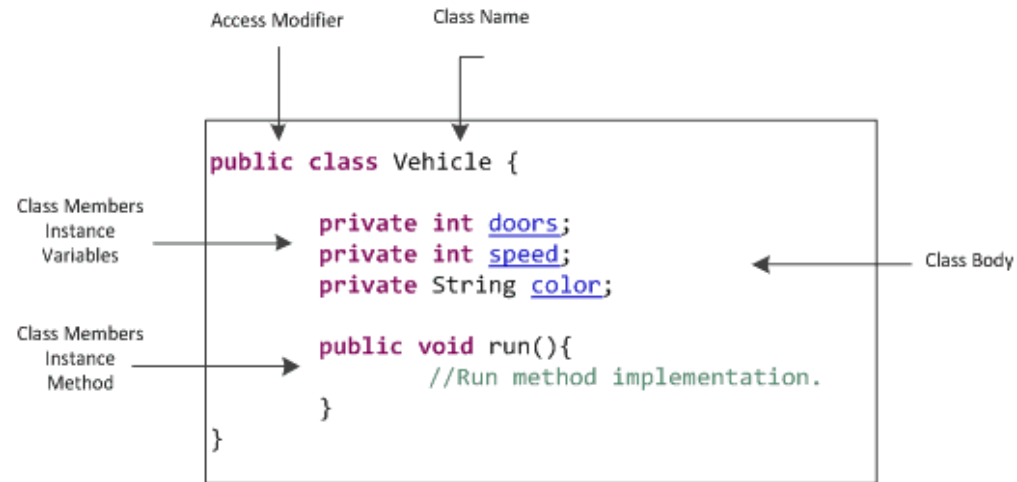
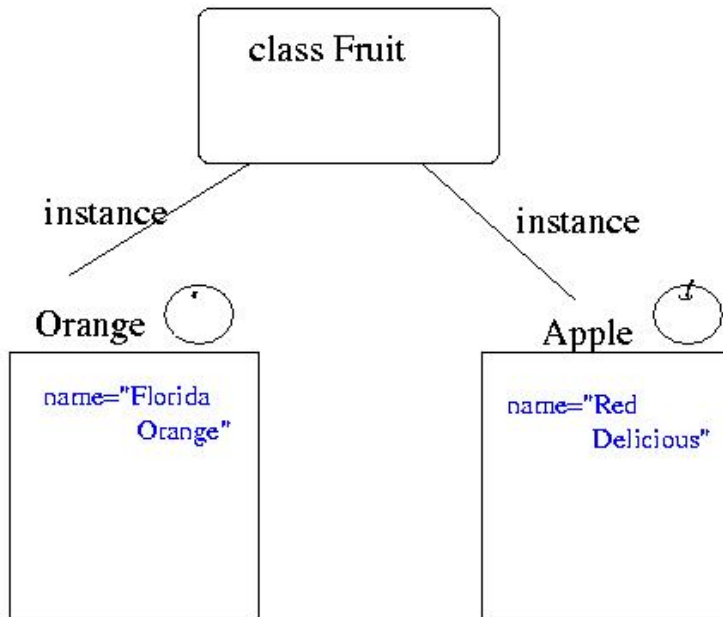
- ❖ The word **behavior** is identical to **function** – it is a piece of code that does something (*or implements a behavior*)
- ❖ The word **state** is identical to **variables** – it is a place to store values within a class.



**When we
assert/define/dec
lare a class
behavior and
state together, it
means that a
class packages
functions and
variables.**

What is instance in Python?

In the Python programming language, an **instance of a class** is also called an **object**. The call will comprise **both data members and methods** and will be accessed by an object of that class.



A class is declared by use of the class keyword. The class body is enclosed between curly braces { and }. The data or variables, defined within a class are called **instance variables**. The code is contained within methods. Collectively, the methods and variables defined within a class are called **members of the class**.

Overview of OOP Terminology

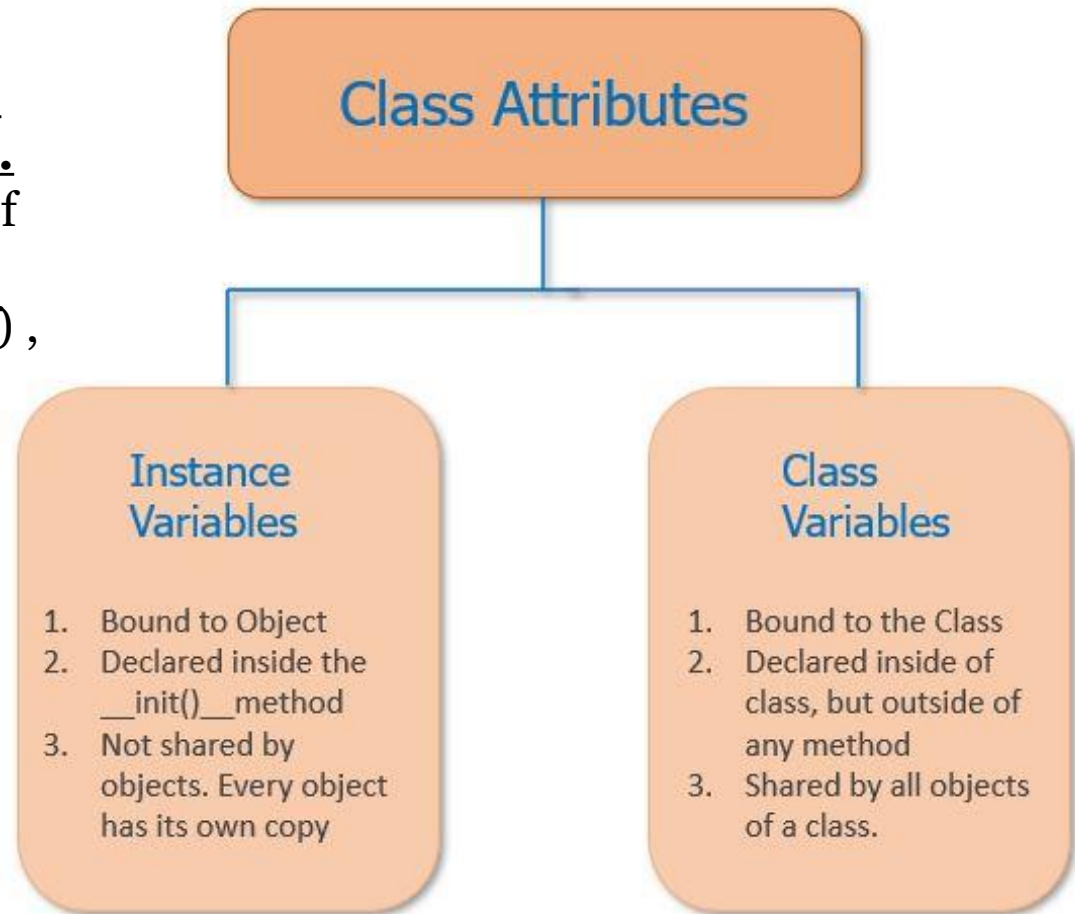
Class is a **user-defined prototype for an object** that defines a set of attributes that characterize any object of the class. The attributes are data members (*class variables and instance variables*) and methods, accessed via dot notation.

- ❖ Class variable is a variable that is shared by all instances of a class. Class variables are defined within a class but outside any of the class's methods. Class variables are not used as frequently as instance variables are.
- ❖ Data member is a class variable or instance variable that holds data associated with a class and its objects.
- ❖ Inheritance is the transfer of the characteristics of a class to other classes that are derived from it.
- ❖ Instance is an individual object of a certain class. An object `obj` that belongs to a class `Circle`, for example, is an instance of the class `Circle`.
- ❖ Instantiation is the creation of an instance of a class.

- ❖ Instance variable is a variable that is defined inside a method and belongs only to the current instance of a class.
- ❖ Object is a unique instance of a data structure that's defined by its class. An object comprises both data members (class variables and instance variables) and methods.
- ❖ Method is a special kind of function that is defined in a class definition.
- ❖ Function overloading is the assignment of more than one behavior to a particular function. The operation performed varies by the types of objects or arguments involved.
- ❖ Operator overloading is the assignment of more than one function to a particular operator.

Class Attributes

A class attribute is a Python variable that belongs to a class rather than a particular object.
It's shared between all the objects of this class and is defined outside the constructor function, `__init__(self)`, of the class.



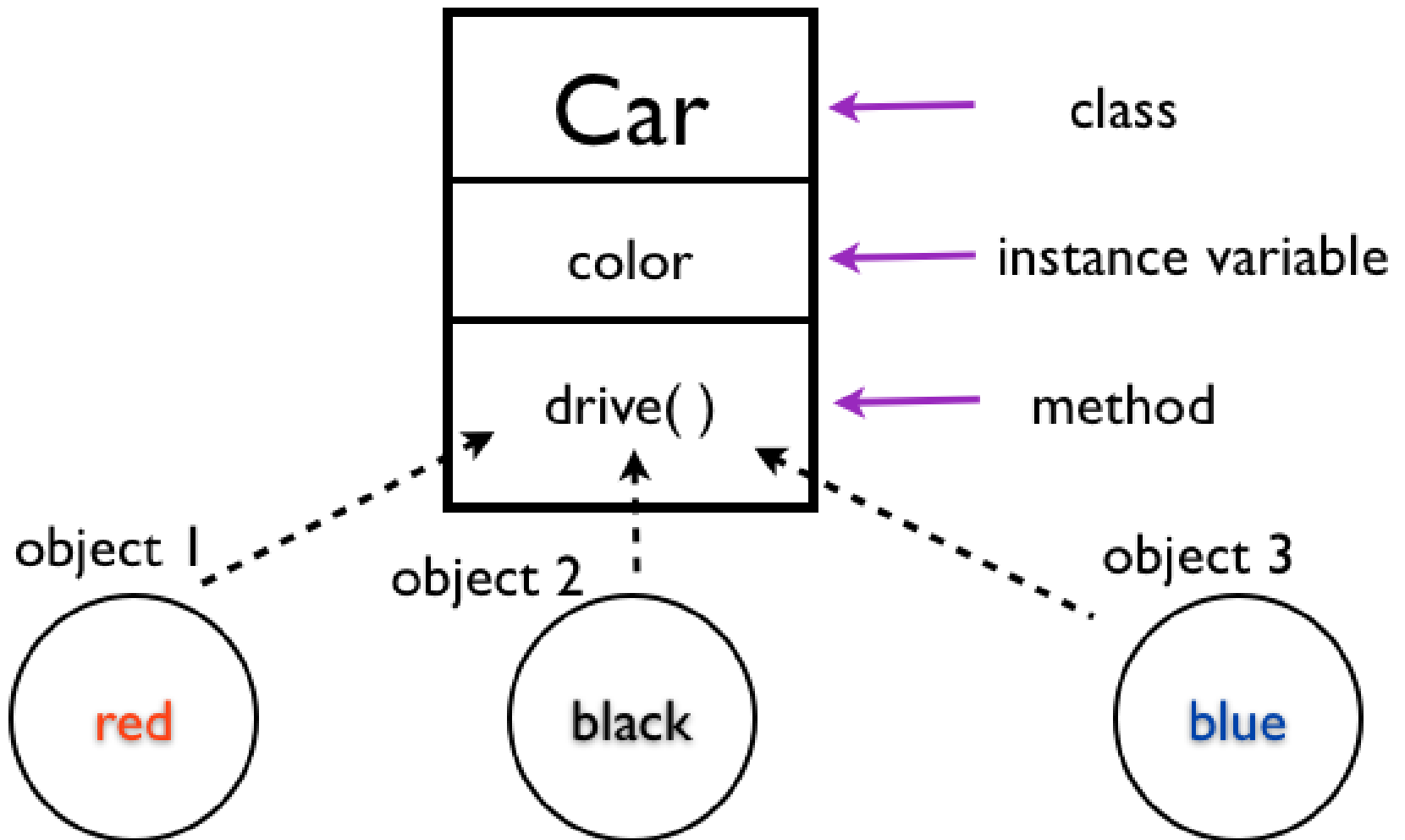
Instance variable is a variable that is defined **inside a method** and belongs only to the current instance of a class.

Classes have methods & attributes

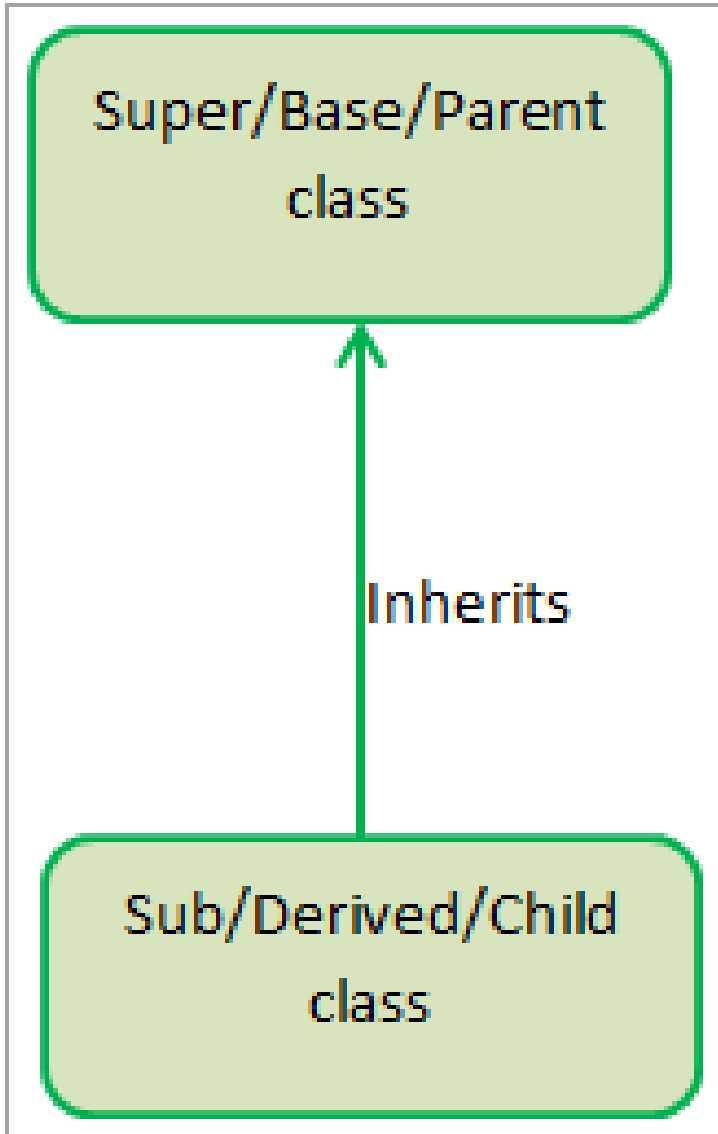
In Python, **creating a method defines a class behavior**. The word method is the OOP name given to a function that is defined within a class.

- 1) **Class functions** is synonym for **methods**.
- 2) **Class variables** is synonym for name **attributes**.
- 3) **Class** is a blueprint/template/design for an instance with exact **behavior**.
- 4) **Object** is one of the instances of the class, perform functionality defined in the class.
- 5) **Type** is indicates the class the instance belongs to.
- 6) **Attribute** is any object value: object.attribute.
- 7) **Method** is a “*callable attribute*” defined in the class.

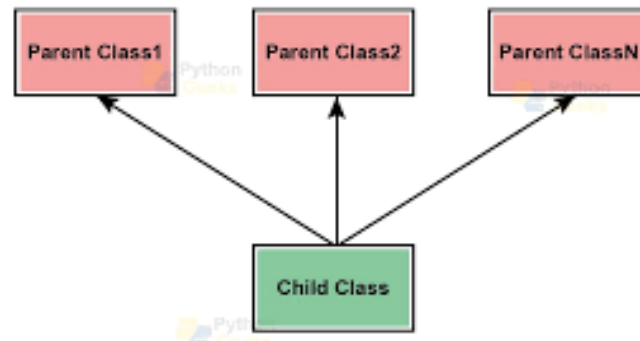
Multiple Objects Share Single Method



Parent Class & Child Class *in* Python



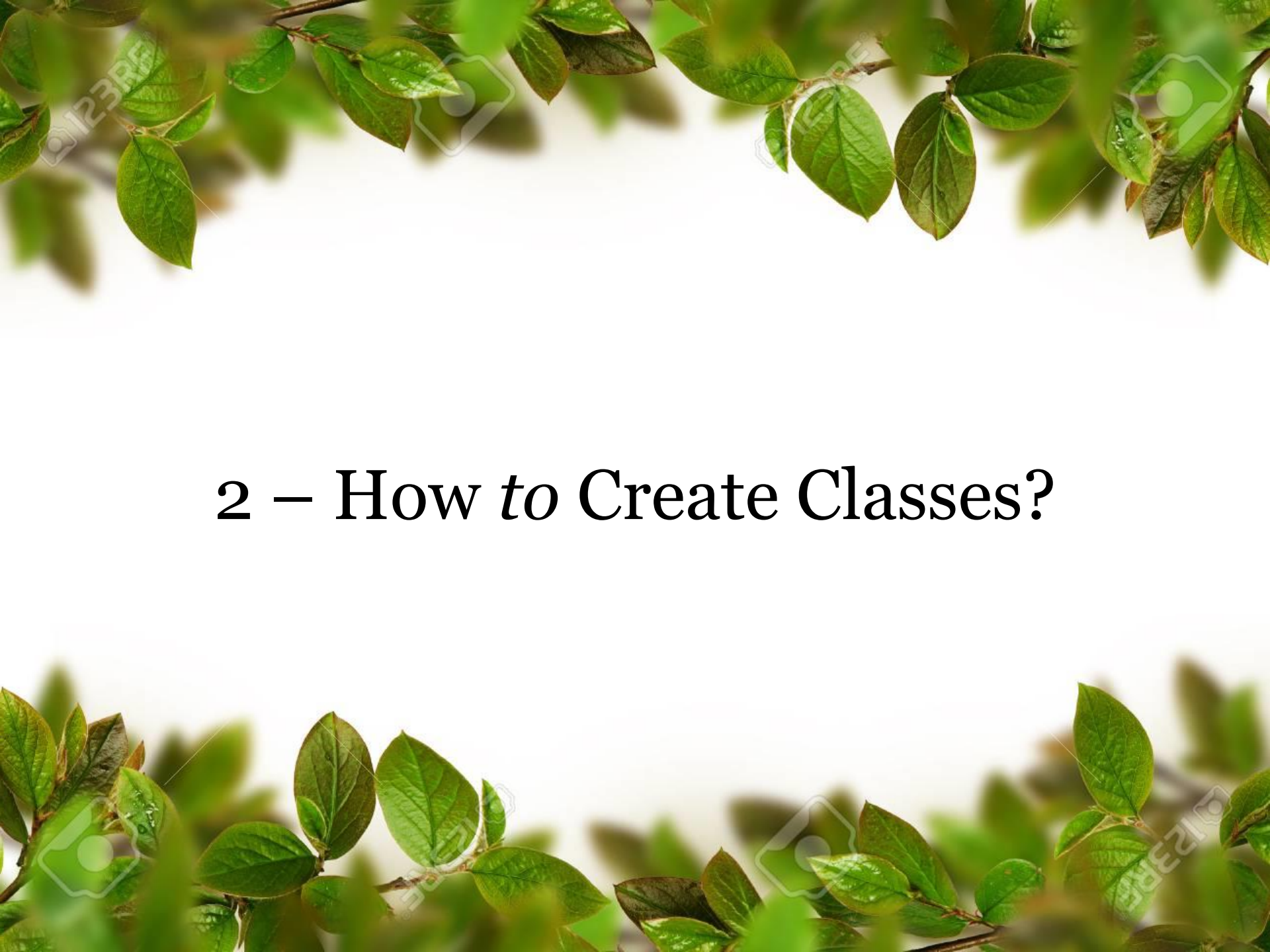
Multiple Inheritance in Python



Python Inheritance:

Inheritance allows us to define a class that inherits all the methods and properties from another class.

- 1) **Parent class** is the class being inherited from, also called **base class**.
- 2) **Child class** is the class that inherits from another class, also called **derived class**.

The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub. The text is centered in the middle of the slide.

2 – How *to* Create Classes?

Creating Classes

Cara membuat sebuah *Class* menggunakan bahasa pemrograman Python sebenarnya cukup mudah, hanya dengan menggunakan kata kunci “**class**” diikuti dengan *nama kelas* dan diakhiri dengan titik dua (:). Umumnya nama dari sebuah *class* OOP akan diawali dengan huruf *capital*.

```
1 class NamaKelas():  
2     """ membuat class """
```


“`def __init__(self)`” artinya?

```
Demo1.py* X
1  # create a class
2  class Produk():
3      # variable
4      jumlah_produk = 0
5
6      # constructor
7  def __init__(self, nama, harga):
8      self.nama = nama
9      self.harga = harga
10     Produk.jumlah_produk += 1
11
12     def jumlah_produk(self):
13         print('Total Produk: ', Produk.jumlah_produk)
14
15     def detail_produk(self):
16         print("Nama : ", self.nama)
17         print("Harga: ", self.harga)
18         print()
```

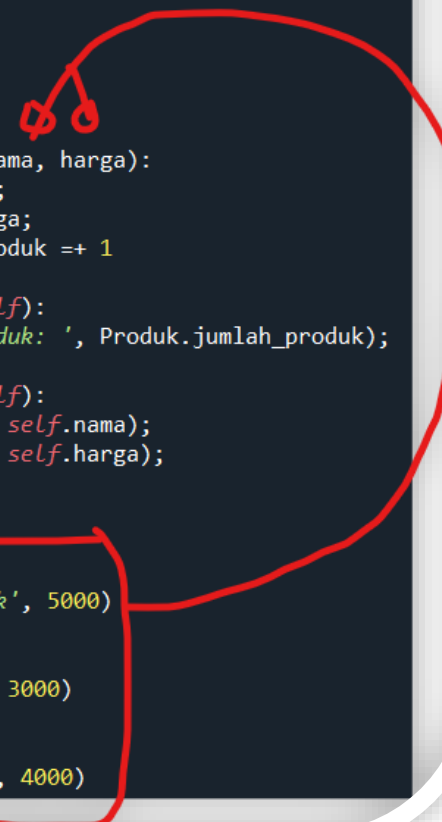
`def __init__(self)` adalah sebuah **constructor function**. Function ini akan dipanggil ketika object baru pertama kali di buat.

Jadi, **constructor** umumnya digunakan untuk menginisialisasi atribut/variable objek.

Def `__init__(self)` adalah metode **constructor** yang menginisialkan pembentukan objek dari kelas.

Instance Object

```
demo1.py X
1  # create a class
2  class Produk():
3      # variable
4      jumlah_produk = 0
5
6      # constructor
7      def __init__(self, nama, harga):
8          self.nama = nama;
9          self.harga = harga;
10         Produk.jumlah_produk += 1
11
12     def jumlah_produk(self):
13         print('Total Produk: ', Produk.jumlah_produk);
14
15     def detail_produk(self):
16         print("Nama : ", self.nama);
17         print("Harga: ", self.harga);
18         print();
19
20
21     # membuat objek pertama
22     produk1 = Produk('Kerupuk', 5000)
23
24     # membuat objek kedua
25     produk2 = Produk('Taro', 3000)
26
27     # membuat objek ketiga
28     produk3 = Produk('Astor', 4000)
```



Untuk membuat **objek instance**, caranya dengan memanggil class kemudian isi argumen sesuai dengan pada **fungsi __init__()** yang sudah kita definisikan sebelumnya.

Mengakses Properties Objek ?

```
demo1.py* X
1 # create a class
2 class Produk():
3     # variable
4     jumlah_produk = 0
5
6     # constructor
7     def __init__(self, nama, harga):
8         self.nama = nama;
9         self.harga = harga;
10
11     def view_jum_produk(self, jumlah):
12         Produk.jumlah_produk = jumlah;
13         print('Total Produk: ', Produk.jumlah_produk);
14
15     def detail_produk(self):
16         print("Nama : ", self.nama);
17         print("Harga: ", self.harga);
18         print();
19
20
21 # membuat objek pertama
22 produk1 = Produk('Kerupuk', 5000)
23
24 # membuat objek kedua
25 produk2 = Produk('Taro', 3000)
26
27 # membuat objek ketiga
28 produk3 = Produk('Astor', 4000)
29
30 # mengakses atribut objek
31 produk1.detail_produk()
32 produk2.detail_produk()
33 produk3.detail_produk()
34
35 produk1.view_jum_produk(5)
36 produk2.view_jum_produk(10)
37 produk3.view_jum_produk(15)
```

Untuk dapat mengakses properties dari objek (**attribute/variable dan juga method**) dengan cara variabel object (.) variabel kelas.

Menurut anda, apakah kita dapat membuat *class* di Python tanpa memiliki konstruktor (*constructor*) atau metode `__init__` ???



Exercise *for* Students

Exercise #1 (Class & Objects)

Spyder (Python 3.9)

File Edit Search Source Run Debug Consoles Projects Tools View Help

TEACHING\CLASS\Object-Oriented Programming\CODES\Lecture5 [Classes]

...\\UNKLAB CLASSES\\UNKLAB TEACHING\\CLASS\\Object-Oriented Programming\\CODES\\Lecture5 [Classes]\\Demo1.py

Demo1.py X

```
1 # create a class
2 class Produk():
3     # variable
4     jumlah_produk = 0
5
6     # method constructor (default method)
7     def __init__(self, nama, harga):
8         self.nama = nama;
9         self.harga = harga;
10
11     # method #1
12     def view_jum_produk(self, jumlah):
13         Produk.jumlah_produk = jumlah;
14         print('Total Produk: ', Produk.jumlah_produk);
15
16     # method #2
17     def detail_produk(self):
18         print("Nama : ", self.nama);
19         print("Harga : ", self.harga);
20
21
22 # membuat objek pertama (Object instantiation)
23 produk1 = Produk('Kerupuk', 5000)
24
25 # membuat objek kedua (Object instantiation)
26 produk2 = Produk('Taro', 3000)
27
28 # membuat objek ketiga (Object instantiation)
29 produk3 = Produk('Astor', 4000)
30
31 # mengakses atribut objek (Accessing class attributes)
32 produk1.detail_produk()
33 produk1.view_jum_produk(5)
34 print("Jumlah produk adalah {}".format(produk1.__class__.jumlah_produk));
35 print("Nama produk adalah {}".format(produk1.nama))
36 print()
37
38 produk2.detail_produk()
39 produk2.view_jum_produk(10)
40 print("Jumlah produk adalah {}".format(produk2.__class__.jumlah_produk));
41 print("Nama produk adalah {}".format(produk2.nama))
42 print()
43
44 produk3.detail_produk()
45 produk3.view_jum_produk(15)
46 print("Jumlah produk adalah {}".format(produk3.__class__.jumlah_produk));
47 print("Nama produk adalah {}".format(produk3.nama))
48
49
```

Perhatikan, terdapat input dari constructor dan dari method parameter.

Console 2/A X

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/Lecture5 [Classes]/Demo1.py', wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/Lecture5 [Classes]')

Nama : Kerupuk
Harga: 5000
Total Produk: 5
Jumlah produk adalah 5
Nama produk adalah Kerupuk

Nama : Taro
Harga: 3000
Total Produk: 10
Jumlah produk adalah 10
Nama produk adalah Taro

Nama : Astor
Harga: 4000
Total Produk: 15
Jumlah produk adalah 15
Nama produk adalah Astor

In [2]:

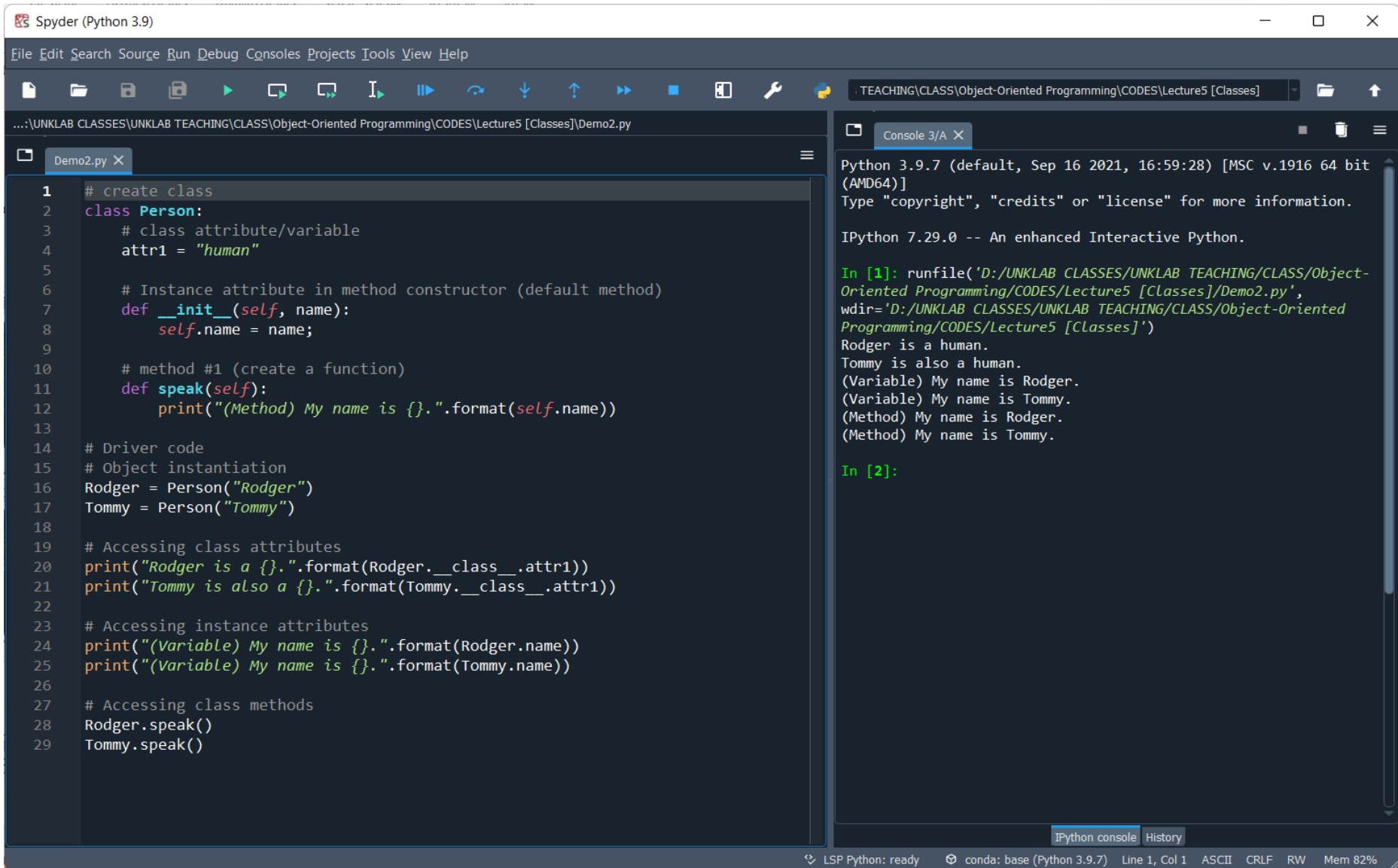
IPython console History

LSP Python: ready conda: base (Python 3.9.7) Line 1, Col 1 ASCII CRLF RW Mem 82%

Modifikasi source code pada file Demo1.py:

1. Dalam **class Produk**, tambahkan 2 *variable* (***tanggal_kedaluwarsa*** dan ***negara_asal***) dengan type String.
tanggal_kedaluwarsa (expired date)
negara_asal (made in Indonesia)
2. Tambahkan dua parameter pada *constructor*.
3. Gunakan *constructor* untuk menginisialisasi dua *variable* yang baru saja ditambahkan.
4. Buat dua method baru (***tampil_expire()*** dan ***tampil_negara_asal()***) dari class Produk. Method-method tersebut dapat mengembalikan nilai berupa String dan tanpa parameter.
5. Tampilkan output dari dua method tersebut menggunakan *object* dari *class*.

Exercise #2 (Class & Objects)



The screenshot displays the Spyder Python IDE interface. The left pane shows a file named `Demo2.py` with the following Python code:

```
1 # create class
2 class Person:
3     # class attribute/variable
4     attr1 = "human"
5
6     # Instance attribute in method constructor (default method)
7     def __init__(self, name):
8         self.name = name;
9
10    # method #1 (create a function)
11    def speak(self):
12        print("(Method) My name is {}".format(self.name))
13
14    # Driver code
15    # Object instantiation
16    Rodger = Person("Rodger")
17    Tommy = Person("Tommy")
18
19    # Accessing class attributes
20    print("Rodger is a {}".format(Rodger.__class__.attr1))
21    print("Tommy is also a {}".format(Tommy.__class__.attr1))
22
23    # Accessing instance attributes
24    print("(Variable) My name is {}".format(Rodger.name))
25    print("(Variable) My name is {}".format(Tommy.name))
26
27    # Accessing class methods
28    Rodger.speak()
29    Tommy.speak()
```

The right pane shows the IPython console output for the executed code:

```
Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/Lecture5 [Classes]/Demo2.py',
wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/Lecture5 [Classes]')
Rodger is a human.
Tommy is also a human.
(Variable) My name is Rodger.
(Variable) My name is Tommy.
(Method) My name is Rodger.
(Method) My name is Tommy.

In [2]:
```

The status bar at the bottom indicates the environment is LSP Python: ready, conda: base (Python 3.9.7), and the current cursor position is Line 1, Col 1. The memory usage is 82%.

Modifikasi source code pada file Demo2.py:

1. Dalam **class Person**, tambahkan 4 *variable* (***age***, ***address***, ***phone_number*** dan ***gender***) dengan type String semua.
2. Tambahkan 4 parameter pada *constructor* untuk menginisialisasi empat *variable* yang baru saja ditambahkan tersebut.
3. Buat 4 method baru (*view_age()*, *view_address()*, *view_phone()* dan *view_gender()*) dari class Person. Method-method tersebut dapat mengembalikan nilai berupa String dan tanpa parameter.
4. Tampilkan output dari ke-empat method tersebut menggunakan object dari class.

Class Activity #1

1. Satu kelompok max 2 students.
2. Jelaskan (*tentang class, variable dan method*) dan buat report dari dua Python files pada exercise di atas.
3. Screenshot output program yang telah dibuat.
4. Submit per student (masing-masing di Google Classroom).
5. Submit report dalam format PDF.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

